Q1 - Benchmarking SATI.c

Dean Yockey

My computer

My personal computer is a Lenovo ThinkCentre I bought for $75 on eBay. It's running the Linux Mint operating system. When running `lscpu`, the first few lines read:

```
Architecture:             x86_64
  CPU op-mode(s):         32-bit, 64-bit
  Address sizes:          39 bits physical, 48 bits virtual
  Byte Order:             Little Endian
CPU(s):                   4
  On-line CPU(s) list:    0-3
```

My computer has 4 CPUs.

SATI.c

I compiled and ran SATI.c after commenting out all printing statements, aside from one for reporting time after the stopwatch stops.

```
if (!id) {
    printf ("T: %8.6f\n", elapsed_time);
    fflush (stdout);
}
```

Benchmarking

I created a bash file for benchmarking. It reads:

```
#! /bin/bash
echo "Q1" > "q1log.txt"
for p in {1..30}
do
    echo "P: $p" >> q1log.txt
    echo "P: $p" # so we can monitor progress
    for i in {1..20}
```

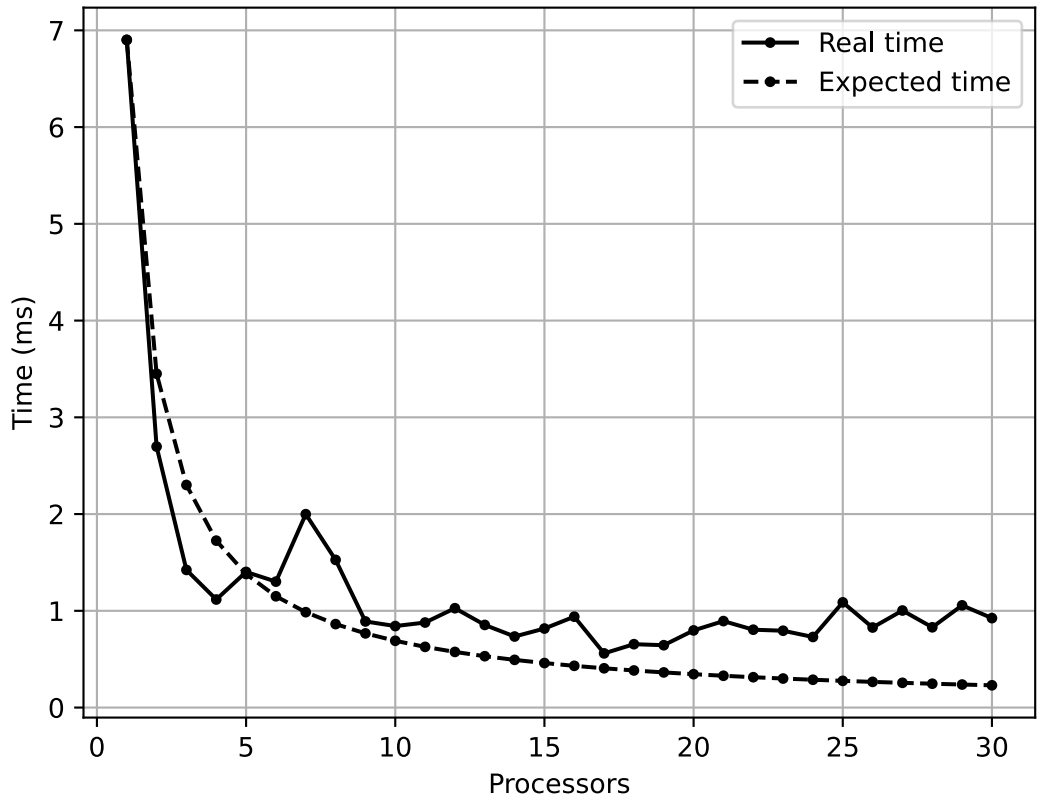
```
    do
        mpirun -n $p --oversubscribe sati >> q1log.txt
    done
done
```

I ran the program 20 times for each process count. The output to `q1log.txt` looked like:

```
Q1
P: 1
T: 0.002704
T: 0.005429
T: 0.009221
T: 0.006354
T: 0.006377
T: 0.009163
T: 0.009185
T: 0.004860
T: 0.009204
T: 0.009196
T: 0.006373
T: 0.003964
T: 0.002604
T: 0.004131
T: 0.009216
T: 0.009195
T: 0.009167
T: 0.003294
T: 0.009216
T: 0.009171
P: 2
T: 0.002593
T: 0.001413
T: 0.002260
T: 0.001501
T: 0.003099
...
```

My results

To extract the results from the log file, and visualize them in a line graph, I wrote a python script. I used regex and matplotlib. This is my graph.



Takeaways

I expected the program to have peak performance at 4 processors, since that's the number of processors my computer actually has, but it actually has the best performance at 17.

At 2, 3, and 4 processors the real results were even better than the idealized expected time. It's hard to guess why. The code is very simple, and all analysis suggests 2 processors should really only be twice as fast. It's possible the `MPI_Reduce` function performs disproportionally slower at just 1 process, but that wouldn't make much sense.

This behavior is consistent across all tests I've performed, but I've simply never been able to justify it.

There's a spike at 7, but from 9 on, the shape of the real time graph is pretty close to the expected time graph.

After the minimum value reached at 17, the real time gradually increases, as a trend. It's slight, but I would assume the time will slowly continue to increase as we go beyond 30.

The fact that the program is optimal at 17, far beyond my real processor count of 4, suggests that MPI's parallelization is very effective even on a single processor.